

Describe el **qué**, no el **cómo**

Hoja de referencia de SWI-Prolog: hechos, reglas, unificación, backtracking, listas, meta-predicados y programación con restricciones (CLP(FD)).

Investigación, redacción y verificación: **MC. René Solís R.**

Origen Marsella/Edimburgo, 1972 **Paradigma** lógico declarativo, resolución SLD

Motor de referencia SWI-Prolog

Verificado con SWI-Prolog 10.0.2 (ago. 2026) — todos los ejemplos ejecutados

§1 Fundamentos del lenguaje

Un programa Prolog no es una secuencia de instrucciones: es una **base de conocimiento** de hechos y reglas. Consultar el programa es plantearle un objetivo que el motor intenta demostrar por **resolución SLD**, probando cláusulas en orden y retrocediendo (*backtracking*) cuando una vía falla.

```
% hechos
progenitor(juan, maria).
progenitor(maria, sofia).

% regla: X es abuelo de Z si X es progenitor de Y, y Y de Z
abuelo(X, Z) :- progenitor(X, Y), progenitor(Y, Z).
```

```
?- abuelo(juan, Quien).
Quien = sofia.
```

El motor **unifica** la consulta con la cabeza de cada cláusula, resuelve el cuerpo de izquierda a derecha y, si algo falla, retrocede al último punto donde había otra opción disponible.

§2 Seis ideas que rompen la intuición

Lo que más sorprende a quien llega desde un lenguaje imperativo o funcional.

01

Cada hecho, regla o consulta en la shell termina con **punto (.)** — como en Erlang, pero aquí el punto separa *cláusulas*, no simplemente expresiones.

02

Las variables (mayúscula inicial o `_`) no se *asignan*: se **unifican**. Un mismo valor puede "deshacerse" si el backtracking regresa sobre ese punto de la prueba.

03

Una consulta no devuelve "un resultado": puede tener **cero, una o muchas soluciones**. Tras la primera, `;` le pide al motor la siguiente por backtracking.

04

El orden importa. El orden de las cláusulas y de los objetivos dentro del cuerpo determina en qué secuencia se exploran las soluciones — a diferencia de la lógica matemática pura, donde el orden es irrelevante.

05

`\+ Objetivo` es **negación como fallo**, no negación clásica: es verdad solo si Prolog *no logra probar* Objetivo con el conocimiento que tiene — no es lo mismo que "Objetivo es falso".

06

El corte `!` no es un valor booleano: es un **efecto de control** que poda el árbol de búsqueda y descarta puntos de retroceso ya visitados, cambiando el conjunto de soluciones que el programa puede encontrar.

§3 Sintaxis esencial

TÉRMINOS

TIPO	EJEMPLO	REGLA
Átomo	<code>juan</code> , <code>'Ana Pérez'</code> , <code>=</code>	minúscula inicial, o entre comillas simples si tiene espacios/mayúsculas
Variable	<code>X</code> , <code>Persona</code> , <code>_Tmp</code> , <code>_</code>	mayúscula inicial o guion bajo; <code>_</code> es la variable anónima
Número	<code>42</code> , <code>3.14</code>	enteros y flotantes
Cadena	<code>"hola"</code>	tipo <i>string</i> nativo en SWI-Prolog
Estructura	<code>punto(X, Y)</code>	functor + argumentos entre paréntesis
Lista	<code>[1, 2, 3]</code> , <code>[H T]</code>	ver §4

OPERADORES DE CLÁUSULA

OPERADOR	SIGNIFICADO
<code>:-</code>	"si" — separa cabeza y cuerpo de una regla (o marca una directiva al inicio de línea)
<code>,</code>	conjunción (Y) entre objetivos
<code>;</code>	disyunción (O) entre objetivos
<code>-></code>	si-entonces, normalmente combinado con <code>;</code>
<code>!</code>	corte — descarta backtracking pendiente

COMENTARIOS

```
% comentario de línea
/* comentario
de varias líneas */
```

§4 Listas

`[]` es la lista vacía; `[Cabeza|Resto]` descompone (o construye) una lista por su primer elemento y la cola.

?- [H|T] = [1, 2, 3].

H = 1, T = [2, 3].

?- length([a, b, c], N).

N = 3.

PREDICADO	HACE
<code>member(X, L)</code>	X pertenece a L (genera X por backtracking si no está ligado)
<code>append(L1, L2, L3)</code>	L3 es la concatenación de L1 y L2 — también divide una lista si L3 es conocida
<code>length(L, N)</code>	N es el número de elementos de L
<code>reverse(L1, L2)</code>	L2 es L1 invertida
<code>nth0(I, L, E)</code> / <code>nth1(I, L, E)</code>	E es el elemento en la posición I, con índice base 0 o base 1
<code>last(L, E)</code>	E es el último elemento de L
<code>sum_list(L, S)</code> / <code>max_list</code> / <code>min_list</code>	suma, máximo o mínimo de una lista de números
<code>msort(L1, L2)</code> / <code>sort(L1, L2)</code>	ordena; <code>sort/2</code> además elimina duplicados
<code>list_to_set(L1, L2)</code>	elimina duplicados preservando el orden
<code>permutation(L1, L2)</code>	L2 es una permutación de L1 (genera todas por backtracking)
<code>select(X, L1, L2)</code>	L2 es L1 sin la primera ocurrencia de X

§5 Control y backtracking

OBJETIVOS PRIMITIVOS

OBJETIVO	COMPORTAMIENTO
<code>true</code>	siempre tiene éxito, sin efecto
<code>fail</code> / <code>false</code>	siempre falla — fuerza backtracking inmediato
<code>repeat</code>	tiene éxito indefinidamente al retroceder — útil para bucles con efectos
<code>\+ G</code>	tiene éxito si G <i>no</i> se puede probar (negación como fallo)

SI-ENTONCES-SI_NO

```
clasificar(N, positivo) :- N > 0, !.  
clasificar(N, cero)    :- N == 0, !.  
clasificar(_, negativo).
```

```
% equivalente con -> ;  
clasificar2(N, R) :-  
  ( N > 0 -> R = positivo  
  ; N == 0 -> R = cero  
  ; R = negativo  
  ).
```

EL CORTE (!)

Al ejecutarse, **descarta las alternativas pendientes** de la cláusula actual y de los objetivos anteriores a él dentro del cuerpo. Se usa para fijar la primera solución encontrada y evitar exploraciones inútiles — pero úsalo con cuidado: puede eliminar soluciones válidas si se coloca mal.

EXCEPCIONES

```
catch(Objetivo, Error, Manejador)  
throw(mi_error(motivo))
```

§6 Meta-predicados

Recolectan o transforman *todas* las soluciones de un objetivo, en vez de una por backtracking.

PREDICADO	HACE
<code>findall(Plantilla, Objetivo, Lista)</code>	todas las soluciones de Objetivo, instanciando Plantilla; lista vacía si no hay ninguna
<code>bagof(Plantilla, Objetivo, Lista)</code>	como findall, pero falla si no hay soluciones y respeta agrupación por variables libres
<code>setof(Plantilla, Objetivo, Lista)</code>	como bagof, pero ordena y elimina duplicados
<code>aggregate_all(count, Objetivo, N)</code>	agrega soluciones: <code>count</code> , <code>sum(E)</code> , <code>bag(E)</code> , <code>set(E)</code> ...
<code>forall(Condicion, Accion)</code>	Accion tiene éxito para <i>cada</i> solución de Condicion (equivalente a un $\forall$)
<code>maplist(Pred, L1, ..., Ln)</code>	aplica Pred elemento a elemento sobre una o varias listas en paralelo
<code>foldl(Pred, L, Acc0, Acc)</code>	reduce la lista L acumulando un estado, como <code>fold</code> en un lenguaje funcional
<code>include(Pred, L, Incluidos)</code> / <code>exclude/3</code>	filtra L según Pred

```
?- include([N]>>(N mod 2 == 0), [1,2,3,4], Pares).
Pares = [2, 4].
```

`maplist/2` exige que el predicado tenga éxito en *todos* los elementos — no filtra, solo verifica; para quedarse con un subconjunto se usa `include/3` (o `exclude/3` para lo contrario).

§7 Aritmética: unificar vs. evaluar

Tres operadores que parecen "igualdad" hacen cosas muy distintas — confundirlos es la fuente de errores más común al empezar.

OPERADOR	QUÉ HACE	EJEMPLO
<code>=</code>	unificación estructural — no evalúa aritmética	<code>X = 2+3</code> liga X al término <code>2+3</code> , no a 5
<code>is</code>	evalúa la expresión aritmética de la derecha y unifica el resultado con la izquierda	<code>X is 2+3</code> liga X a 5
<code>==</code> / <code>=\=</code>	compara el <i>valor</i> evaluado de dos expresiones	<code>2+3 == 6-1</code> → true
<code>==</code> / <code>\==</code>	compara <i>identidad estructural</i> de términos, sin evaluar ni unificar	<code>2+3 == 5</code> → false
<code><</code> <code>></code> <code><=</code> <code>>=</code>	comparación numérica tras evaluar ambos lados	<code>3*2 > 5</code> → true

`succ(X, Y)` ($Y = X+1$) y `plus(X, Y, Z)` ($Z = X+Y$) son útiles porque, a diferencia de `is`, pueden usarse en más de una dirección.

§8 Átomos, cadenas y E/S

PREDICADO	HACE
<code>atom_concat(A1, A2, A3)</code>	concatena átomos (o los separa por backtracking)
<code>atom_string(Atomo, Str)</code>	convierte entre átomo y string
<code>atom_length(A, N)</code>	longitud de un átomo
<code>sub_atom(A, Antes, Long, Despues, Sub)</code>	extrae o busca subcadenas de un átomo
<code>number_string(N, Str)</code>	convierte entre número y string
<code>write(T)</code> / <code>writeln(T)</code>	imprime un término (con o sin salto de línea)
<code>format(Fmt, Args)</code>	impresión con plantilla: <code>~w</code> término, <code>~a</code> átomo, <code>~d</code> entero, <code>~n</code> salto de línea

```
?- format("~a tiene ~d años~n", [ana, 23]).
ana tiene 23 años
```

§9 Base de conocimiento dinámica

Un programa puede modificar sus propios hechos en tiempo de ejecución — útil para memoización, agentes o simulaciones.

```
:- dynamic visto/1.

asserta(visto(inicio)). % añade al principio
assertz(visto(fin)).   % añade al final
retract(visto(inicio)). % elimina la primera coincidencia
retractall(visto(_)).  % elimina todas las coincidencias
```

Todo predicado que se vaya a modificar con `assert` / `retract` debe declararse antes con `:- dynamic Nombre/Aridad.`

§10 CLP(FD): restricciones sobre enteros

`library(clpfd)` añade **programación con restricciones** sobre dominios finitos: en vez de calcular un valor, se describen las restricciones que debe cumplir y se deja que el motor busque asignaciones válidas.

```
:- use_module(library(clpfd)).
```


SÍMBOLO / PREDICADO	SIGNIFICADO
<code>#=</code> / <code>#\=</code>	igualdad / desigualdad como restricción (sustituye a <code>is</code> y <code>:=</code> sobre enteros, y puede resolver en ambas direcciones)
<code>#<</code> <code>#></code> <code>#=<</code> <code>#>=</code>	comparaciones como restricción
<code>X in 1..10</code>	restringe el dominio de X al rango
<code>[X,Y,Z] ins 1..10</code>	restringe el dominio de varias variables a la vez
<code>all_different(Lista)</code> / <code>all_distinct(Lista)</code>	todas las variables toman valores distintos entre sí
<code>sum(Lista, #=: N)</code>	la suma de la lista debe ser N
<code>label(Lista)</code> / <code>labeling(Opciones, Lista)</code>	fuerza la enumeración de valores concretos que satisfacen las restricciones acumuladas

EJEMPLO: SEND + MORE = MONEY

```
resolver(Digitos) :-
  Digitos = [S,E,N,D,M,O,R,Y],
  Digitos ins 0..9,
  all_different(Digitos),
  S #\= 0, M #\= 0,
  1000*S + 100*E + 10*N + D
  + 1000*M + 100*O + 10*R + E
  #= 10000*M + 1000*O + 100*N + 10*E + Y,
  label(Digitos).
```

Este mismo mecanismo — declarar restricciones y dejar que `label/1` busque — es el que resuelve N-reinas, Sudoku o problemas de horarios sin escribir un algoritmo de búsqueda a mano.

§11 Referencia rápida

CONSULTAR EN LA SHELL

TECLA / COMANDO	EFEECTO
<code>.</code> + Enter	ejecuta la consulta
<code>;</code>	pide la siguiente solución por backtracking
<code>a</code>	en modo de más soluciones, muestra todas de golpe
<code>[archivo].</code> / <code>consult(archivo).</code>	carga un archivo <code>.pl</code>
<code>halt.</code>	sale de la shell

OPERADORES COMUNES

SÍMBOLO	USO
<code>:-</code>	regla / directiva
<code>, ; -></code>	Y, O, si-entonces
<code>!</code>	corte
<code>\+</code>	negación como fallo
<code>= \= == \==</code>	unificación / no-unificación / identidad / no-identidad
<code>is := =\= < > =< >=</code>	evaluación y comparación aritmética
<code>[H T]</code>	descomposición de listas
<code>#= #\= in ins</code>	restricciones CLP(FD)

hechos + reglas

unificación

backtracking

negación como fallo

restricciones (CLP)

Investigación, redacción y verificación técnica de esta hoja: **MC. René Solís R.** Fuentes: documentación oficial de SWI-Prolog — [Control Predicates](#), [library\(lists\)](#), [library\(clpfd\)](#). — y [Learn Prolog Now!](#) ([lpn.swi-prolog.org](#)). Todos los fragmentos de código fueron ejecutados en [SWI-Prolog 10.0.2](#) (la versión estable descargable en [swi-prolog.org](#) al momento de esta edición) para confirmar sintaxis y salidas, incluyendo el resolutor CLP(FD) del ejemplo SEND+MORE=MONEY (solución verificada: S=9 E=5 N=6 D=7 M=1 O=0 R=8 Y=2).